

Composition

A local-first, AI-assisted previs studio that turns a director's intent into an editable 3D scene, a camera guide, and optionally a generated video.

The product in one sentence

Instead of asking a video model to guess staging and motion from a text prompt, Composition gives creators a controllable intermediate representation: a lightweight 3D scene and timeline that can be reviewed before generation.

The end-to-end creative loop

1. Stage a mannequin and composite props. 2. Keyframe or apply reusable motion blocks. 3. Frame and animate a virtual camera. 4. Export a reviewable guide video. 5. Add direction and selected appearance references. 6. Submit an intentionally scoped video job and keep its result locally.

Why this is useful to a creator

Problem	Design response	Judge takeaway
Text-to-video is hard to steer	Editable scene, camera and animation guide precede generation	Control is explicit, inspectable and reversible
AI changes can be unsafe or vague	Proposal -> validate -> preview -> approve -> atomic apply	The model never silently mutates the scene
Generation is expensive and asynchronous	Persistent job IDs, saved inputs and restart recovery	Avoids accidental duplicate paid submissions
Phone filming is disconnected from previs	ARKit phone becomes a live virtual camera controller	Physical camera language can drive a digital shot

Scope statement

This is a single-user local studio, not a cloud collaboration product. It intentionally favors a clear, testable local workflow over unsupported promises such as automatic collision avoidance, new-rig generation, or autonomous final-editing.

One canonical scene, several focused boundaries

The project document is the source of truth. Rendering, UI state, provider responses and persisted files are deliberately kept separate from it.

Layer	Technical components	Responsibility
Browser client	React 19, Vite, TypeScript, Three.js	Editor UI, 3D viewport, timeline, local recovery draft, guide capture
Core domain	Project v2, sampling, clips, velocity, proposal validation	Pure scene data and deterministic pose evaluation
Local server	Express 5, Zod, filesystem store, FFmpeg	API validation, asset/project persistence, video conversion, job recovery
AI boundary	Gemini + fal clients	Structured proposals, image references, Hunyuan motion, Seedance output
Phone companion	Swift, SwiftUI, ARKit, WKWebView	World-tracked pose capture and returned virtual-shot monitor

Canonical data model

Project v2 contains objects, keyframe tracks, optional animation clips, optional velocity curves, a virtual camera, and generation metadata. It stores plain JSON only - never renderer instances - so it can autosave, export/import, validate on the server and survive renderer changes.

Animation model

A track is identified by (**objectId, target, channel**). Targets are the object model or a humanoid joint; channels are position, rotation and scale. The editor samples a pose at any time through base keys or a clip, easing, velocity remapping, per-axis value handles, and quaternion-aware rotation paths. This keeps AI, manual editing and export on the same motion semantics.

Important design choices

Choice	Why it matters
30 fps timeline with 2-10 s scenes	Frame-aligned editing keeps keys predictable while meeting output constraints.
Bottom-center pivots; meters; Y-up	A stable spatial convention shared by UI, Three.js, API and ARKit mapping.
Animation blocks are self-contained	A block can be cached, moved, stretched or split without mutating its source library asset.
React store is UI state; Project is domain state	Undo, selection and playhead do not leak into portable scene files.

ARKit-to-editor handoff: realtime control without cloud video

The iPhone companion treats the phone as a physical camera controller. It sends ARKit pose data to the local editor; the editor returns a rendered virtual shot. Phone camera imagery is never uploaded.

Transport map

Hop	Protocol / payload	Purpose
Pair	Loopback REST -> QR session	Create expiring, single-sender pairing state
Phone -> relay	Local WebSocket on port 3003	Bounded ARKit pose/control messages; up to 30 Hz
Relay -> browser	Same-origin Server-Sent Events	Session status and state updates without exposing LAN socket to browser
Editor -> phone preview	JPEG baseline or WebRTC via protected receiver page	Return a virtual-shot monitor, not the physical camera feed
WebRTC signaling	Native socket plus REST/SSE	SDP/ICE remains inside the paired native path; receiver does not open a LAN WebSocket

Why both WebSockets and SSE?

WebSockets fit the phone's bidirectional low-latency pose/control stream. SSE fits the browser's same-origin event subscription and keeps the browser off the LAN-facing tracking socket. This separation narrows the exposure of a prototype designed for a trusted local network.

Pose-to-keyframe pipeline

ARKit position and quaternion orientation arrive in a right-handed, Y-up coordinate system. The editor validates monotonic sequence/time, aligns the phone to the virtual scene, resamples poses to 30 fps, converts them to continuous Euler rotations, and writes one undoable camera block. A gap over 350 ms, stale data over 500 ms, tracking loss, disconnect or hidden editor ends the usable recording rather than silently bridging broken tracking.

Latency experiment: what we can honestly claim

Measured/implemented instrumentation	Target, not a result
20-pulse run exports samples, timeout count, median/p95, capture/copy timing, receiver stats and browser details. Pulse is control-to-decoded-frame estimate on the phone clock.	Wi-Fi criterion: ≥ 50 received fps in 60 fps mode; median < 100 ms; p95 < 150 ms; desktop receive-to-render < 30 ms.
WebRTC is 960x540 targeting 60 or 30 fps. JPEG baseline is 480x270 at about 6 fps. Pose transport is up to 30 Hz.	Physical motion-to-photon latency remains pending; it requires a high-frame-rate camera test.

Guardrails make an AI workflow demonstrable

Composition treats model output and long-running providers as untrusted boundaries. Validation happens before mutation, and persistent records make failures inspectable rather than invisible.

Area	Implemented control	Judge-ready explanation
AI proposals	Schema validation, scene signature, pure preview, atomic apply	A proposal is rejected if stale, malformed, incompatible or outside scene rules; accept creates one undoable edit.
Paid generation	Application ID saved before submission; input fingerprint; no automatic provider resubmit	A lost response cannot quietly create a second paid job. Same ID + same input returns the existing job.
Recovery	Job/provider IDs persist; server resumes polling after restart	Completed output bytes live in local asset storage, not an expiring provider URL.
Storage	Atomic JSON writes, content-addressed assets	SHA-256 of role + bytes deduplicates assets; one server process per data directory avoids write races.
Credentials	Server-only .env keys; loopback API and origin checks	Keys never ship through Vite client variables. Public hosting would need auth and tenant storage.
Phone session	Expiring QR code, one sender, limits/rate checks	Current local WebSocket is intentionally documented as unencrypted and for trusted LANs only.

Concrete system limits

Category	Limits
Scene	1 humanoid; up to 32 objects; 2-10 second editing timeline; 30 fps; up to 64 clips
Video/output	4-10 s whole-duration output; default 5 s, 720p, 16:9, audio off; 50 MB guide; 100 MB output
References	Up to 24 generated images saved per project; up to 9 unique references submitted; 30 MB per reference
Phone protocol	30 Hz pose cap; >350 ms packet gap or >500 ms stale pose stops capture; 70 KB WebSocket message ceiling; non-signaling messages 4 KB

Verification evidence

The repository includes Vitest unit/server coverage plus Playwright browser workflows. Documented focused verification after the QR update: **99 unit/server tests passed**, web and native builds passed, and **10 focused phone browser tests passed**. An earlier full browser run recorded 49 passes and 5 failures; it was not rerun for that update and is not claimed green. Automated tests use mocked providers and synthetic poses, so they validate integration and persistence - not paid-provider quality or physical-device latency.

Questions to expect - and the crisp technical answer

Likely question	Answer
What is technically novel here?	The core is the controllable handoff: natural-language intent or physical camera motion becomes a validated, editable scene representation before video generation. We separate creative planning, deterministic staging, and expensive rendering.
Why not send one prompt directly to a video model?	Text-only generation collapses camera, blocking, motion and appearance into one opaque request. Our guide video gives the model a concrete motion/framing contract, while the scene remains editable and reviewable.
How do you stop AI from breaking the scene?	Every AI proposal is typed, validated against the current project, previewed on a copy, and applied atomically only after approval. A scene signature prevents applying a stale proposal.
How does the phone handoff work?	ARKit sends pose/control messages over a paired local WebSocket. The desktop maps and resamples them to a 30 fps camera block, then sends back a virtual preview using JPEG or experimental WebRTC. Browser session events use SSE.
Do you have latency results?	We have instrumented a repeatable 20-pulse benchmark and defined targets, but we do not claim physical-device latency yet. The correct next measurement films phone motion and desktop response together with a high-frame-rate camera.
What happens if generation fails or the app restarts?	The job and input fingerprint are persisted before provider submission. Jobs with a provider ID resume polling after restart; uncertain submissions are never automatically resent, avoiding duplicate charges.
What are the current limitations?	Single-user/local storage, trusted-LAN unencrypted phone socket, one humanoid, box/composite props, no collision avoidance, pending live paid-provider acceptance, and pending physical-device latency acceptance.

Suggested live demo narrative

1. Place a simple composite prop with Director or manual tools.
2. Add a motion block and show it remains editable.
3. Use the phone to frame/record a virtual camera move.
4. Export the guide and explain that this is the reviewable handoff to generation.
5. Show the persistent job/output state and call out the duplicate-charge guardrail.

One closing line

“Composition makes AI video direction less like rolling dice and more like directing a shot: plan it, inspect it, adjust it, then generate it.”

Source basis: repository README, docs/api.md, docs/phone-camera.md, docs/phone-latency-experiment.md, and implementation contracts in src/core and server/. Metrics labeled targets or pending are intentionally not presented as achieved results.